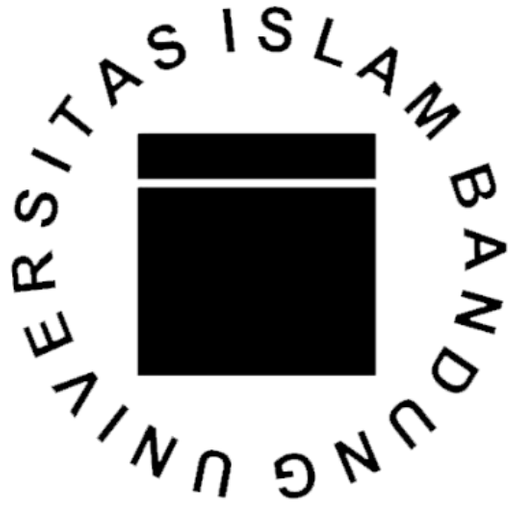
Pemodelan Regresi Logistik Menggunakan *Pyspark*

Disusun dalam Rangka Memenuhi Tugas Ujian Akhir Semester Mata Kuliah Big Data Analytics

Semester Genap Tahun Akademik 2025/2026



Dosen Pengampu:

Abdul Kudus, S.SI., M.SI., PH.D.

Disusun Oleh:

Keysha Jauzi Althaff 10060122019

PROGRAM STUDI STATISTIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM (MIPA)

UNIVERSITAS ISLAM BANDUNG

2025

SOAL
[CPMK01, CPMK02; Skor 100]

Diberikan data "cancer_tumor.csv" pada link
http://bit.ly/44rUgX9

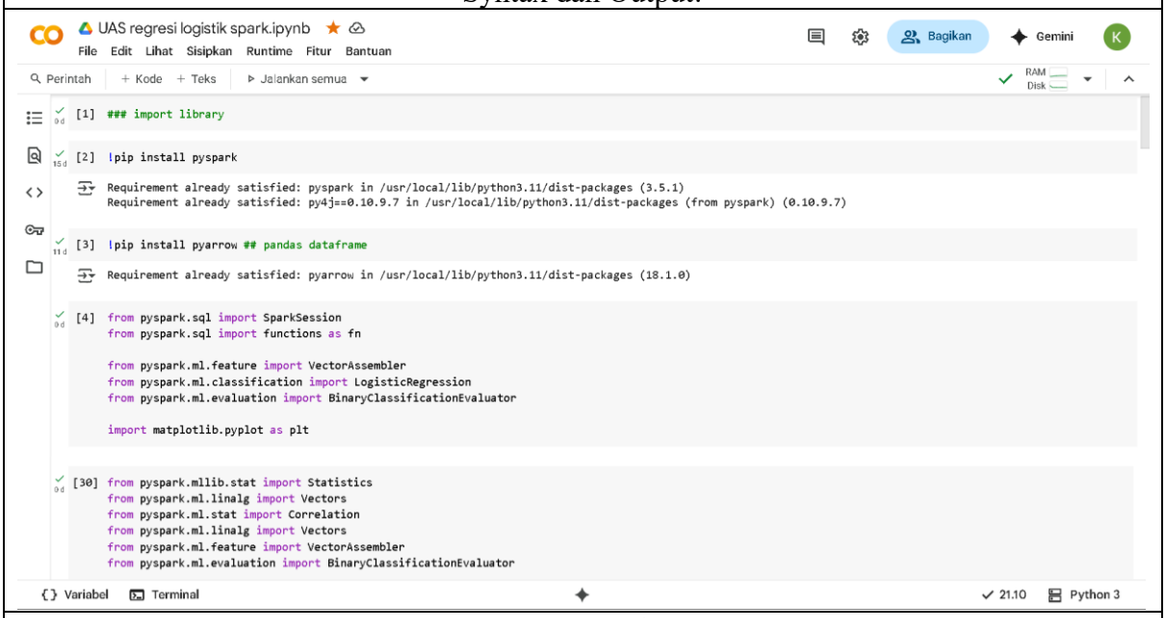
Lakukan pemodelan regresi logistik dengan tools big data pyspark dengan tahapan sebagai berikut:

1. **(CPMK 02 Skor 5)** Import library yang diperlukan.
2. **(CPMK 02 Skor 5)** Mulai dengan spark session.
3. **(CPMK 02 Skor 5)** Baca data.
4. **(CPMK 02 Skor 5)** Menetapkan variabel respon
   $'diagnosis' = \begin{cases} 1, & \text{jika M (malignant)} \\ 0, & \text{jika B (benign)} \end{cases}$
5. **(CPMK 02 Skor 5)** Menetapkan variabel prediktor: 'radius_mean', 'texture_mean', 'area_mean', 'perimeter_mean', 'compactness_mean', 'radius_se', 'area_se', 'perimeter_se', 'radius_worst', 'texture_worst', 'area_worst', 'concavity_worst', 'symmetry_worst'
6. **(CPMK 02 Skor 10)** Split data menjadi 2 bagian dengan komposisi:
   $d\text{ata} = \begin{cases} 80\%, & \text{sebagai data training} \\ 20\%, & \text{sebagai data testing} \end{cases}$
7. **(CPMK 03 Skor 20)** Lakukan pendugaan model regresi logistik menggunakan data training dengan iterasi maksimum 1000.
8. **(CPMK 03 Skor 20**) Berapa tingkat akurasi model dugaannya berdasarkan nilai Area Under Curve (AUC)?
9. **(CPMK 03 Skor 25)** Lakukan evaluasi terhadap model dugaan dengan menggunakan data testing. Berapa tingkat akurasinya?

PENYELESAIAN

1. Import library yang diperlukan.

Syntax dan Output:

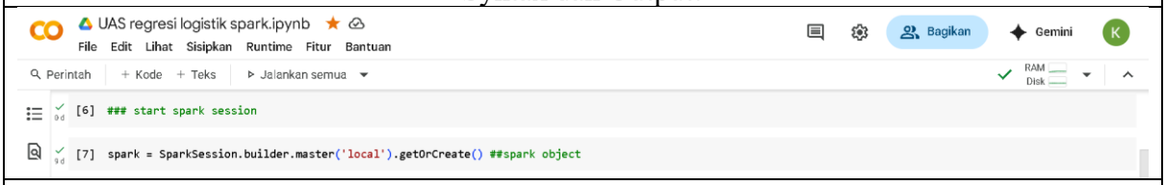


Interpretasi:

Library PySpark diimpor untuk melakukan pemodelan regresi logistik, mulai dari pengolahan data (SparkSession, functions, VectorAssembler), pembangunan model (LogisticRegression), hingga evaluasi performa model (BinaryClassificationEvaluator). Selain itu, matplotlib.pyplot digunakan untuk visualisasi kurva ROC.

2. Mulai dengan spark session.

Syntax dan Output:

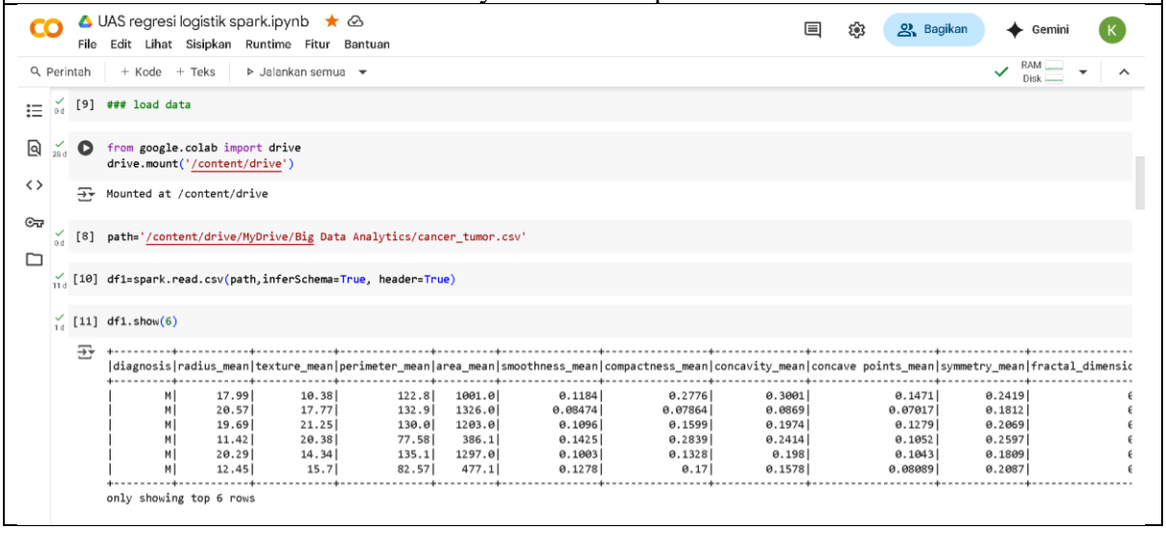


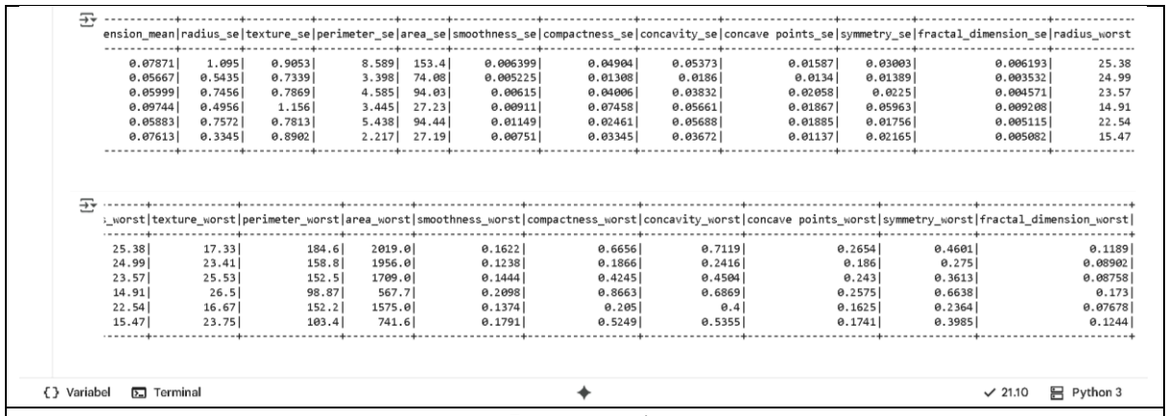
Interpretasi:

Spark session dimulai dengan SparkSession.builder.master('local').getOrCreate() untuk membuat objek Spark yang digunakan dalam pemrosesan data dan pemodelan di PySpark.

3. Baca data

Syntax dan Output:





```
ension_mean|radius_se|texture_se|perimeter_se|area_se|smoothness_se|compactness_se|concavity_se|concave points_se|symmetry_se|fractal_dimension_se|radius_worst
0.07871|   1.095|   0.9053|      8.589|  153.4|    0.006399|     0.04904|     0.05373|      0.01587|     0.03003|             0.006193|        25.38
0.05667|  0.5435|   0.7339|      3.398|  74.08|    0.005225|     0.01308|      0.0186|       0.0134|     0.01389|             0.003532|        24.99
0.05999|  0.7456|   0.7869|      4.585|  94.03|     0.00615|     0.04006|     0.03832|      0.02058|      0.0225|             0.004571|        23.57
0.09744|  0.4956|    1.156|      3.445|  27.23|     0.00911|     0.07458|     0.05661|      0.01867|     0.05963|             0.009208|        14.91
0.05883|  0.7572|   0.7813|      5.438|  94.44|     0.01149|     0.02461|     0.05688|      0.01885|     0.01756|             0.005115|        22.54
0.07613|  0.3345|   0.8902|      2.217|  27.19|     0.00751|     0.03345|     0.03672|      0.01137|     0.02165|             0.005082|        15.47

_worst|texture_worst|perimeter_worst|area_worst|smoothness_worst|compactness_worst|concavity_worst|concave points_worst|symmetry_worst|fractal_dimension_worst|
 25.38|        17.33|          184.6|    2019.0|          0.1622|           0.6656|           0.7119|              0.2654|        0.4601|                 0.1189|
 24.99|        23.41|          158.8|    1956.0|          0.1238|           0.1866|           0.2416|               0.186|         0.275|                0.08902|
 23.57|        25.53|          152.5|    1709.0|          0.1444|           0.4245|           0.4504|               0.243|        0.3613|                0.08758|
 14.91|         26.5|          98.87|     567.7|          0.2098|           0.8663|           0.6869|              0.2575|        0.6638|                 0.173|
 22.54|        16.67|          152.2|    1575.0|          0.1374|            0.205|              0.4|              0.1625|        0.2364|                0.07678|
 15.47|        23.75|          103.4|     741.6|          0.1791|           0.5249|           0.5355|              0.1741|        0.3985|                 0.1244|
```

Variabel Terminal 21.10 Python 3

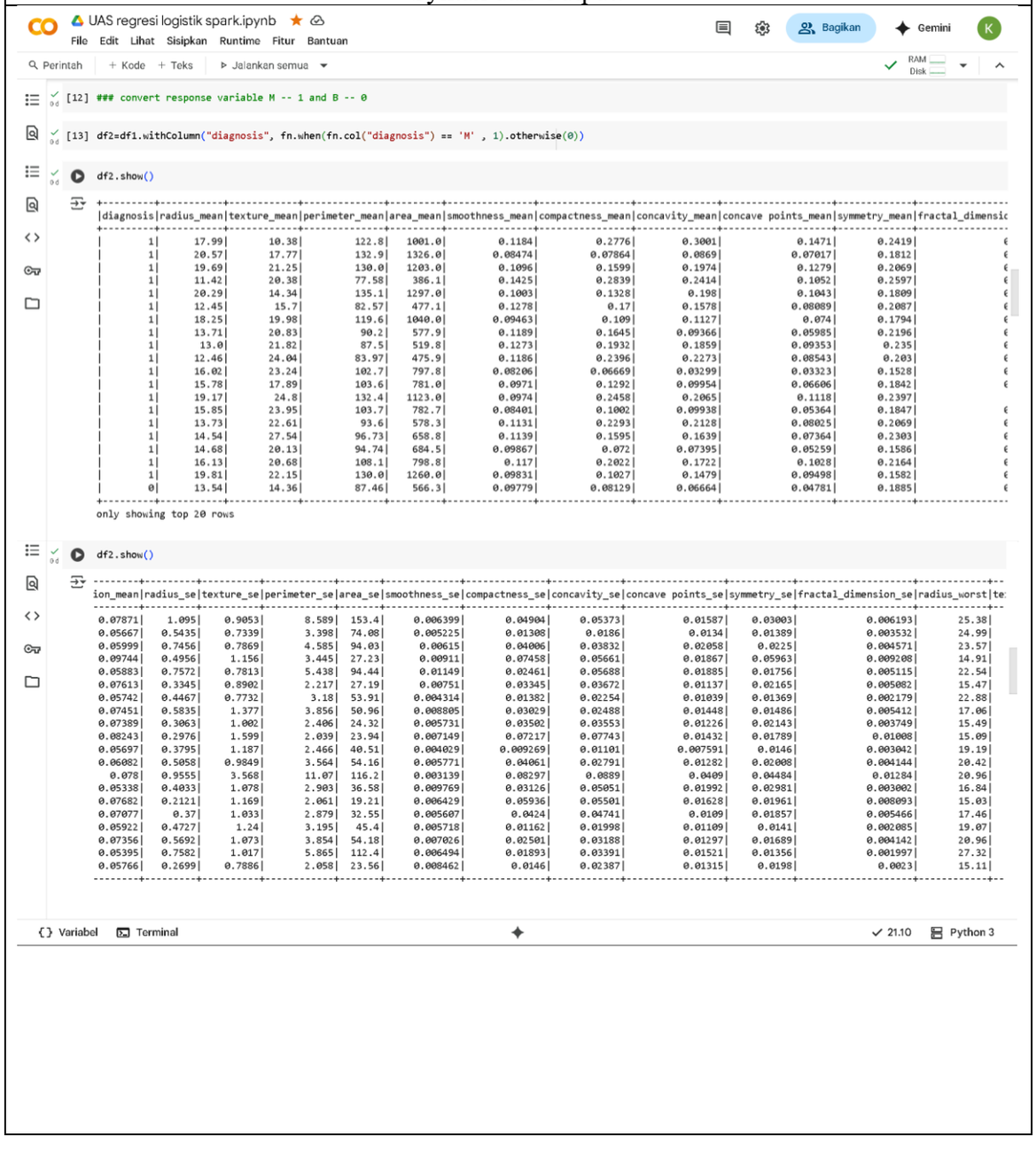
Interpretasi:

Dataset diunggah dari Google Drive menggunakan drive.mount, kemudian dibaca ke dalam PySpark DataFrame menggunakan spark.read.csv dengan opsi inferSchema=True dan header=True untuk mengenali tipe data dan nama kolom secara otomatis

4. Menetapkan variabel respon

$$'diagnosis' = \begin{cases} 1, & \text{jika M (malignant)} \\ 0, & \text{jika B (benign)} \end{cases}$$

Syntax dan Output:



UAS regresi logistik spark.ipynb

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Bagikan Gemini

Perintah + Kode + Teks Jalankan semua

RAM Disk

```
[12] ### convert response variable M -- 1 and B -- 0

[13] df2=df1.withColumn("diagnosis", fn.when(fn.col("diagnosis") == 'M' , 1).otherwise(0))

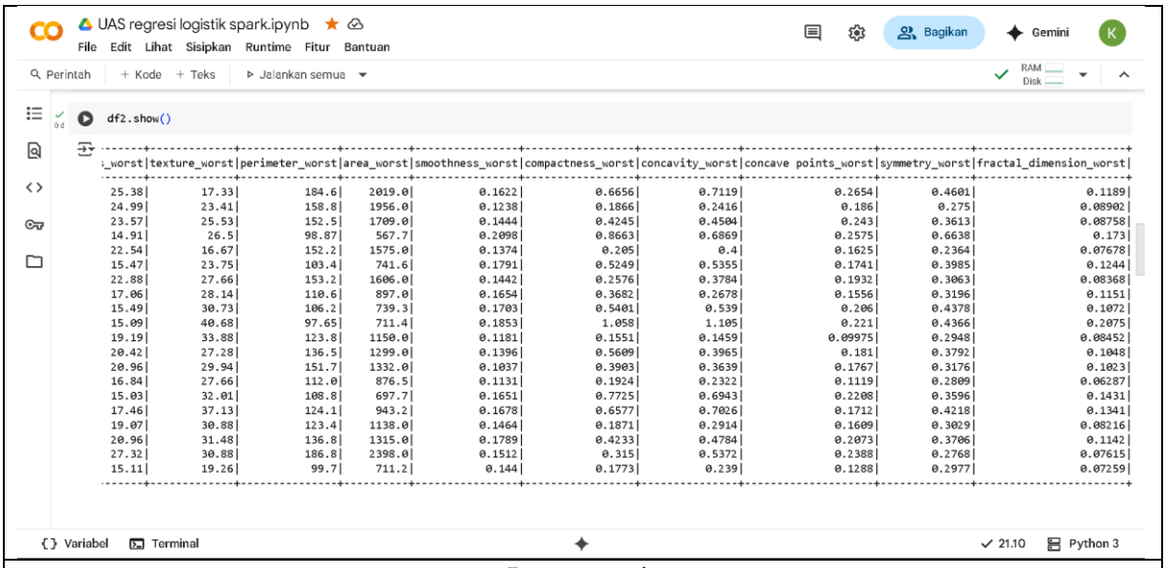
df2.show()
```

```
|diagnosis|radius_mean|texture_mean|perimeter_mean|area_mean|smoothness_mean|compactness_mean|concavity_mean|concave points_mean|symmetry_mean|fractal_dimensic
|        1|      17.99|       10.38|         122.8|   1001.0|         0.1184|          0.2776|        0.3001|             0.1471|       0.2419|
|        1|      20.57|       17.77|         132.9|   1326.0|        0.08474|         0.07864|        0.0869|            0.07017|       0.1812|
|        1|      19.69|       21.25|         130.0|   1203.0|         0.1096|          0.1599|        0.1974|             0.1279|       0.2069|
|        1|      11.42|       20.38|          77.58|    386.1|         0.1425|          0.2839|        0.2414|             0.1052|       0.2597|
|        1|      20.29|       14.34|         135.1|   1297.0|         0.1003|          0.1328|         0.198|             0.1043|       0.1809|
|        1|      12.45|        15.7|          82.57|    477.1|         0.1278|            0.17|        0.1578|            0.08089|       0.2087|
|        1|      18.25|       19.98|         119.6|   1040.0|        0.09463|           0.109|        0.1127|               0.074|       0.1794|
|        1|      13.71|       20.83|          90.2|    577.9|         0.1189|          0.1645|       0.09366|            0.05985|       0.2196|
|        1|       13.0|       21.82|          87.5|    519.8|         0.1273|          0.1932|        0.1859|            0.09353|       0.235|
|        1|      12.46|       24.04|         83.97|    475.9|         0.1186|          0.2396|        0.2273|            0.08543|       0.203|
|        1|      16.02|       23.24|         102.7|    797.8|        0.08206|         0.06669|       0.03299|            0.03323|       0.1528|
|        1|      15.78|       17.89|         103.6|    781.0|         0.0971|          0.1292|       0.09954|            0.06606|       0.1842|
|        1|      19.17|        24.8|         132.4|   1123.0|         0.0974|          0.2458|        0.2065|             0.1118|       0.2397|
|        1|      15.85|       23.95|         103.7|    782.7|        0.08401|          0.1002|       0.09938|            0.05364|       0.1847|
|        1|      13.73|       22.61|          93.6|    578.3|         0.1131|          0.2293|        0.2128|            0.08025|       0.2069|
|        1|      14.54|       27.54|          96.73|    658.8|         0.1139|          0.1595|        0.1639|            0.07364|       0.2303|
|        1|      14.68|       20.13|          94.74|    684.5|        0.09867|           0.072|       0.07395|            0.05259|       0.1586|
|        1|      16.13|       20.68|          108.1|    798.8|          0.117|          0.2022|        0.1722|             0.1028|       0.2164|
|        1|      19.81|       22.15|         130.0|   1260.0|        0.09831|          0.1027|        0.1479|            0.09498|       0.1582|
|        0|      13.54|       14.36|          87.46|    566.3|        0.09779|         0.08129|       0.06664|            0.04781|       0.1885|
only showing top 20 rows
```

```
df2.show()
```

```
ion_mean|radius_se|texture_se|perimeter_se|area_se|smoothness_se|compactness_se|concavity_se|concave points_se|symmetry_se|fractal_dimension_se|radius_worst|te
 0.07871|    1.095|    0.9053|       8.589|  153.4|     0.006399|      0.04904|      0.05373|       0.01587|     0.03003|              0.006193|       25.38|
 0.05667|   0.5435|    0.7339|       3.398|  74.08|     0.005225|      0.01308|       0.0186|        0.0134|     0.01389|              0.003532|       24.99|
 0.05999|   0.7456|    0.7869|       4.585|  94.03|      0.00615|      0.04006|      0.03832|       0.02058|      0.0225|              0.004571|       23.57|
 0.09744|   0.4956|     1.156|       3.445|  27.23|      0.00911|      0.07458|      0.05661|       0.01867|     0.05963|              0.009208|       14.91|
 0.05883|   0.7572|    0.7813|       5.438|  94.44|      0.01149|      0.02461|      0.05688|       0.01885|     0.01756|              0.005115|       22.54|
 0.07613|   0.3345|    0.8902|       2.217|  27.19|      0.00751|      0.03345|      0.03672|       0.01137|     0.02165|              0.005082|       15.47|
 0.05742|   0.4467|    0.7732|        3.18|  53.91|     0.004314|      0.01382|      0.02254|       0.01039|     0.01369|              0.002179|       22.88|
 0.07451|   0.5835|     1.377|       3.856|  50.96|     0.008805|      0.03029|      0.02488|       0.01448|     0.01486|              0.005412|       17.06|
 0.07389|   0.3063|     1.002|       2.406|  24.32|     0.005731|      0.03502|      0.03553|       0.01226|     0.02143|              0.003749|       15.49|
 0.08243|   0.2976|     1.599|       2.039|  23.94|     0.007149|      0.07217|      0.07743|       0.01432|     0.01789|               0.01008|       15.09|
 0.05697|   0.3795|     1.187|       2.466|  40.51|     0.004029|     0.009269|      0.01101|      0.007591|      0.0146|              0.003042|       19.19|
 0.06082|   0.5058|    0.9849|       3.564|  54.16|     0.005771|      0.04061|      0.02791|       0.01282|     0.02008|              0.004144|       20.42|
   0.078|   0.9555|     3.568|       11.07|  116.2|     0.003139|      0.08297|       0.0889|        0.0409|     0.04484|               0.01284|       20.96|
 0.05338|   0.4033|     1.078|       2.903|  36.58|     0.009769|      0.03126|      0.05051|       0.01992|     0.02981|              0.003002|       16.84|
 0.07682|   0.2121|     1.169|       2.061|  19.21|     0.006429|      0.05936|      0.05501|       0.01628|     0.01961|              0.008093|       15.03|
 0.07077|     0.37|     1.033|       2.879|  32.55|     0.005607|       0.0424|      0.04741|        0.0109|     0.01857|              0.005466|       17.46|
 0.05922|   0.4727|      1.24|       3.195|   45.4|     0.005718|      0.01162|      0.01998|       0.01109|      0.0141|              0.002085|       19.07|
 0.07356|   0.5692|     1.073|       3.854|  54.18|     0.007026|      0.02501|      0.03188|       0.01297|     0.01689|              0.004142|       20.96|
 0.05395|   0.7582|     1.017|       5.865|  112.4|     0.006494|      0.01893|      0.03391|       0.01521|     0.01356|              0.001997|       27.32|
 0.05766|   0.2699|    0.7886|       2.058|  23.56|     0.008462|       0.0146|      0.02387|       0.01315|      0.0198|               0.0023|       15.11|
```

Variabel Terminal 21.10 Python 3

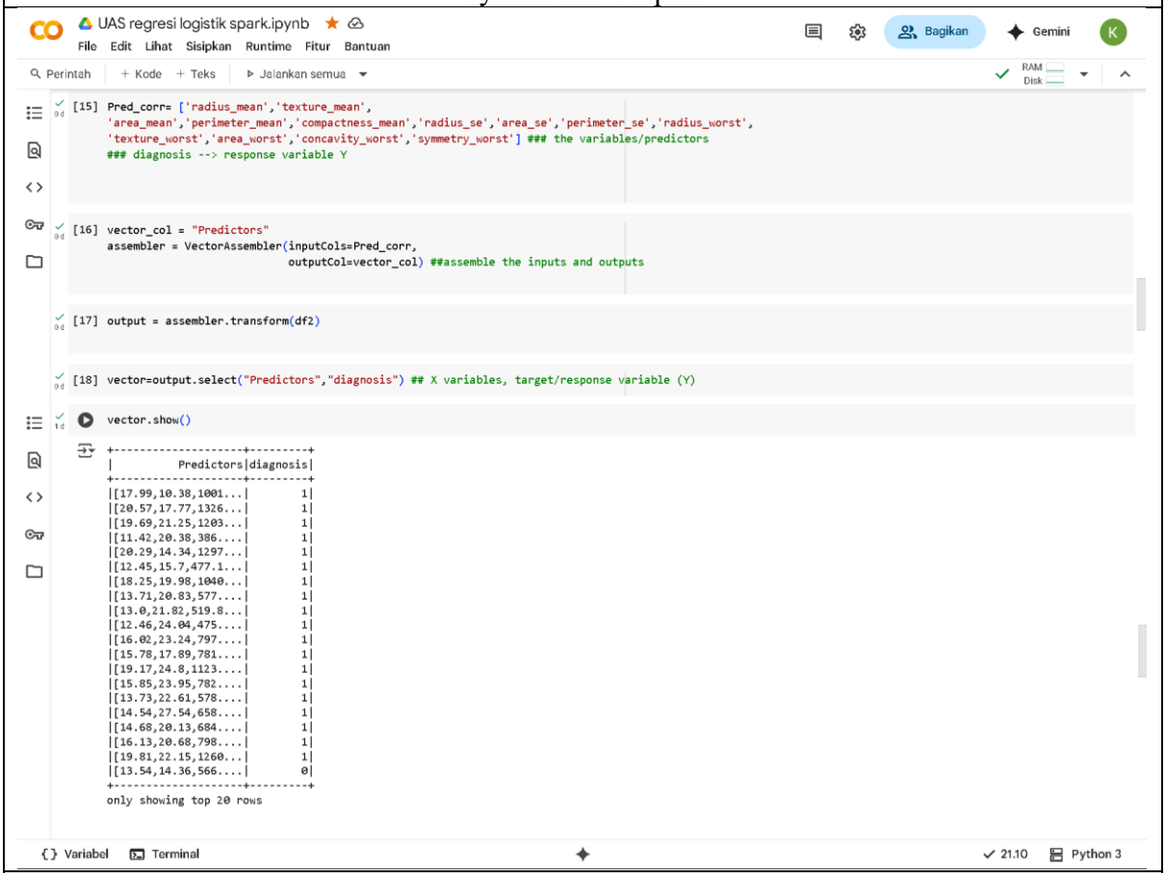


Interpretasi:

Variabel respon diagnosis dikonversi menjadi numerik, yaitu 1 untuk kasus malignan (M) dan 0 untuk benign (B), menggunakan fungsi when dari PySpark.

5. Menetapkan variabel prediktor: 'radius_mean', 'texture_mean', 'area_mean', 'perimeter_mean', 'compactness_mean', 'radius_se', 'area_se', 'perimeter_se', 'radius_worst', 'texture_worst', 'area_worst', 'concavity_worst', 'symmetry_worst'

Syntax dan Output:



Interpretasi:

Sebanyak 13 variabel dipilih sebagai prediktor, lalu digabungkan menjadi satu vektor fitur menggunakan VectorAssembler. Output berupa dua kolom: Predictors sebagai vektor fitur (X) dan diagnosis sebagai variabel respon (Y), yang siap digunakan untuk pemodelan regresi logistic

6. Split data menjadi 2 bagian dengan komposisi:

$$data = \begin{cases} 80\%, & \text{sebagai data training} \\ 20\%, & \text{sebagai data testing} \end{cases}$$

Syntax dan Output:



Interpretasi:

Data dibagi menjadi dua bagian menggunakan randomSplit, dengan 80% data digunakan untuk pelatihan (training) dan 20% sisanya untuk pengujian (testing).

7. Lakukan pendugaan model regresi logistik menggunakan data training dengan iterasi maksimum 1000.

Syntax dan Output:

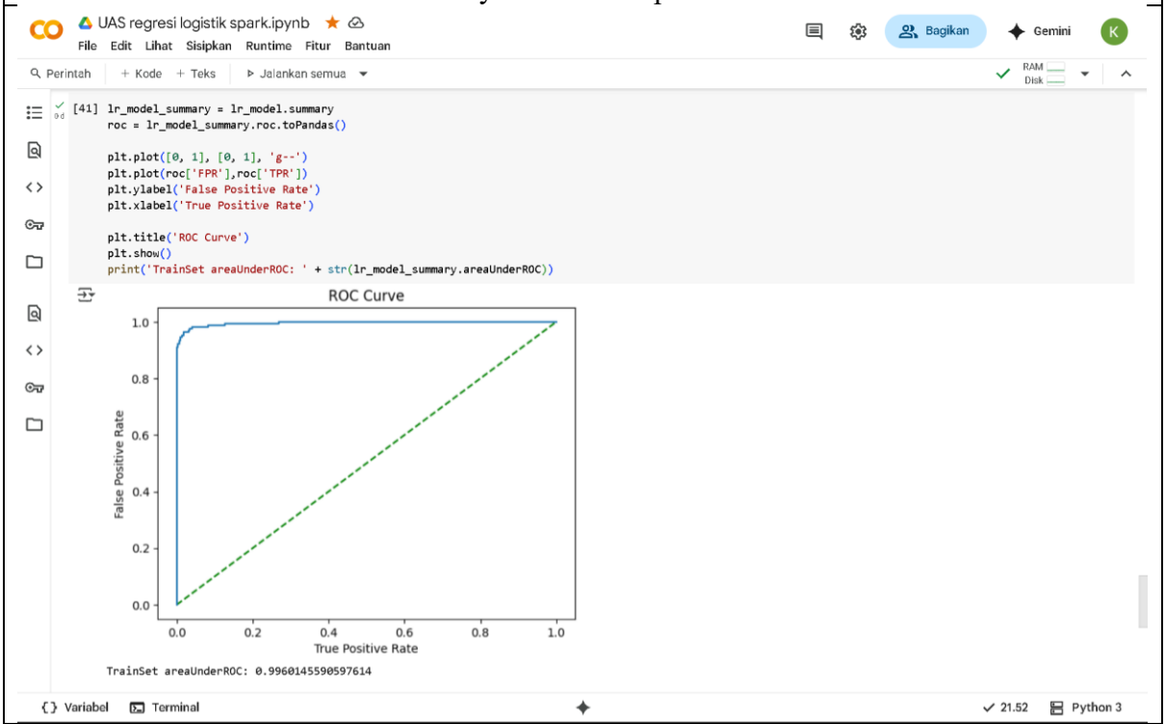


Interpretasi:

Model regresi logistik diduga menggunakan data training dengan parameter maxIter=1000 untuk menetapkan jumlah maksimum iterasi dalam proses optimasi, menggunakan LogisticRegression dari PySpark ML.

8. Berapa tingkat akurasi model dugaannya berdasarkan nilai Area Under Curve (AUC)?

Syntax dan Output:

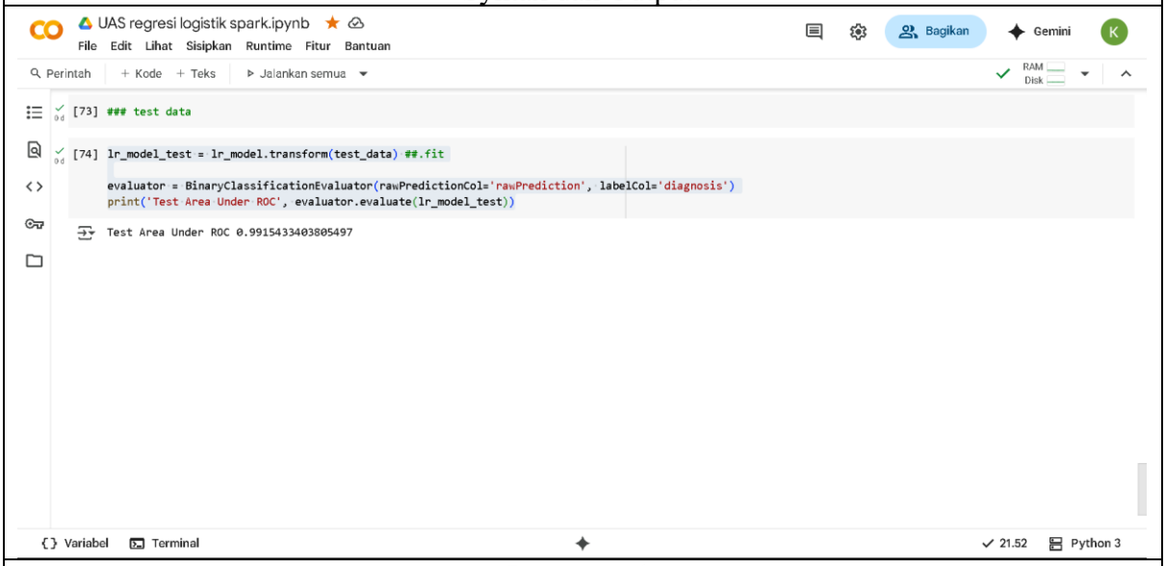


Interpretasi:

Berdasarkan hasil plot ROC Curve, diperoleh nilai Area Under Curve (AUC) sebesar 0,9960. Nilai ini menunjukkan bahwa model regresi logistik memiliki kemampuan klasifikasi yang sangat baik dalam membedakan antara tumor malignan dan benign pada data pelatihan.

9. Lakukan evaluasi terhadap model dugaan dengan menggunakan data testing. Berapa tingkat akurasinya?

Syntax dan Output:



Interpretasi:

Evaluasi model dilakukan menggunakan data testing, dan diperoleh nilai Area Under ROC (AUC) sebesar 0,9915. Nilai ini menunjukkan bahwa model memiliki tingkat akurasi yang sangat tinggi dan tetap konsisten dalam membedakan antara tumor malignan dan benign meskipun diuji pada data yang belum pernah dilihat sebelumnya.

LAMPIRAN


```
### import library
```

```
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.11/dist-packages (3
Requirement already satisfied: py4j==0.10.9.7 in /usr/local/lib/python3.11/dist-pack
```

```
!pip install pyarrow ## pandas dataframe
```

```
Requirement already satisfied: pyarrow in /usr/local/lib/python3.11/dist-packages (1
```

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as fn

from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator

import matplotlib.pyplot as plt
```

```
from pyspark.mllib.stat import Statistics
from pyspark.ml.linalg import Vectors
from pyspark.ml.stat import Correlation
from pyspark.ml.linalg import Vectors
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.evaluation import BinaryClassificationEvaluator
```

```
### start spark session
```

```
spark = SparkSession.builder.master('local').getOrCreate() ##spark object
```

```
### load data
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.
```

```
path='/content/drive/MyDrive/Big Data Analytics/cancer_tumor.csv'
```

```
df1=spark.read.csv(path,inferSchema=True, header=True)
```

```
df1.show(6)
```

```
+---------+----------+------------+--------------+---------+---------------+-------
|diagnosis|radius_mean|texture_mean|perimeter_mean|area_mean|smoothness_mean|compact
+---------+----------+------------+--------------+---------+---------------+-------
|        M|      17.99|       10.38|         122.8|   1001.0|         0.1184|
|        M|      20.57|       17.77|         132.9|   1326.0|        0.08474|
|        M|      19.69|       21.25|         130.0|   1203.0|         0.1096|
|        M|      11.42|       20.38|         77.58|    386.1|         0.1425|
|        M|      20.29|       14.34|         135.1|   1297.0|         0.1003|
|        M|      12.45|        15.7|         82.57|    477.1|         0.1278|
+---------+----------+------------+--------------+---------+---------------+-------
only showing top 6 rows
```

```
### convert response variable M -- 1 and B -- 0
```

```
df2=df1.withColumn("diagnosis", fn.when(fn.col("diagnosis") == 'M' , 1).otherwise(0))
```

```
df2.show()
```

```
+---------+----------+-----------+-------------+---------+--------------+-------
|diagnosis|radius_mean|texture_mean|perimeter_mean|area_mean|smoothness_mean|compact
+---------+----------+-----------+-------------+---------+--------------+-------
|        1|     17.99|      10.38|        122.8|   1001.0|        0.1184|
|        1|     20.57|      17.77|        132.9|   1326.0|       0.08474|
|        1|     19.69|      21.25|        130.0|   1203.0|        0.1096|
|        1|     11.42|      20.38|        77.58|    386.1|        0.1425|
|        1|     20.29|      14.34|        135.1|   1297.0|        0.1003|
|        1|     12.45|       15.7|        82.57|    477.1|        0.1278|
|        1|     18.25|      19.98|        119.6|   1040.0|       0.09463|
|        1|     13.71|      20.83|         90.2|    577.9|        0.1189|
|        1|      13.0|      21.82|         87.5|    519.8|        0.1273|
|        1|     12.46|      24.04|        83.97|    475.9|        0.1186|
|        1|     16.02|      23.24|        102.7|    797.8|       0.08206|
|        1|     15.78|      17.89|        103.6|    781.0|        0.0971|
|        1|     19.17|       24.8|        132.4|   1123.0|        0.0974|
|        1|     15.85|      23.95|        103.7|    782.7|       0.08401|
|        1|     13.73|      22.61|         93.6|    578.3|        0.1131|
|        1|     14.54|      27.54|        96.73|    658.8|        0.1139|
|        1|     14.68|      20.13|        94.74|    684.5|       0.09867|
|        1|     16.13|      20.68|        108.1|    798.8|         0.117|
|        1|     19.81|      22.15|        130.0|   1260.0|       0.09831|
|        0|     13.54|      14.36|        87.46|    566.3|       0.09779|
+---------+----------+-----------+-------------+---------+--------------+-------
only showing top 20 rows
```

```
Pred_corr= ['radius_mean','texture_mean',
'area_mean','perimeter_mean','compactness_mean','radius_se','area_se','perimeter_se','ra
'texture_worst','area_worst','concavity_worst','symmetry_worst'] ### the variables/predi
### diagnosis --> response variable Y
```

```
vector_col = "Predictors"
assembler = VectorAssembler(inputCols=Pred_corr,
                            outputCol=vector_col) ##assemble the inputs and outputs
```

```
output = assembler.transform(df2)
```

```
vector=output.select("Predictors","diagnosis") ## X variables, target/response variable
```

```
vector.show()
```

```
+--------------------+---------+
|          Predictors|diagnosis|
+--------------------+---------+
|[17.99,10.38,1001...|        1|
|[20.57,17.77,1326...|        1|
|[19.69,21.25,1203...|        1|
|[11.42,20.38,386....|        1|
```

```
|[20.29,14.34,1297...|        1|
|[12.45,15.7,477.1...|        1|
|[18.25,19.98,1040...|        1|
|[13.71,20.83,577....|        1|
|[13.0,21.82,519.8...|        1|
|[12.46,24.04,475....|        1|
|[16.02,23.24,797....|        1|
|[15.78,17.89,781....|        1|
|[19.17,24.8,1123....|        1|
|[15.85,23.95,782....|        1|
|[13.73,22.61,578....|        1|
|[14.54,27.54,658....|        1|
|[14.68,20.13,684....|        1|
|[16.13,20.68,798....|        1|
|[19.81,22.15,1260...|        1|
|[13.54,14.36,566....|        0|
+--------------------+---------+
only showing top 20 rows
```

```
###split data into training and testing
train_data,test_data=vector.randomSplit([0.8,0.2]) ##80% for training
```

```
regressor=LogisticRegression(featuresCol="Predictors",labelCol="diagnosis",maxIter=1000)
```

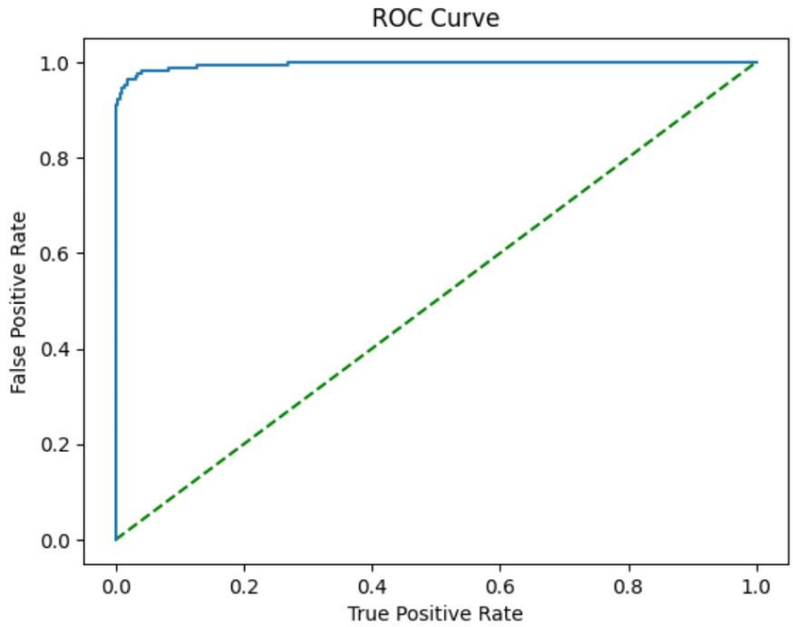
```
##predict on training dataset
lr_model = regressor.fit(train_data) ##.fit
```

```
### plotting area under curve (AUC) as accuracy level
```

```
lr_model_summary = lr_model.summary
roc = lr_model_summary.roc.toPandas()

plt.plot([0, 1], [0, 1], 'g--')
plt.plot(roc['FPR'],roc['TPR'])
plt.ylabel('False Positive Rate')
plt.xlabel('True Positive Rate')

plt.title('ROC Curve')
plt.show()
print('TrainSet areaUnderROC: ' + str(lr_model_summary.areaUnderROC))
```




```
TrainSet areaUnderROC: 0.9960145590597614
```

```
### test data
```

```
lr_model_test = lr_model.transform(test_data) ##.fit

evaluator = BinaryClassificationEvaluator(rawPredictionCol='rawPrediction', labelCol='diag
print('Test Area Under ROC', evaluator.evaluate(lr_model_test))
```